Protocolo Experimental

Ponto 3 - Persistência local e funcionamento offline
Grupo 6
- Guilherme Sousa 55010
- Henrique Marques 55129

1. Enquadramento Teórico

A persistência de dados local serve para guardar informações no próprio telemóvel. Assim, quando o utilizador fecha a aplicação ou fica sem acesso à Internet, os dados não se perdem.
No Android, existem diferentes formas de guardar dados:

- **SharedPreferences / DataStore:** Para guardar definições simples (como o modo escuro ou opções do utilizador).
- **Ficheiros internos:** Para guardar ficheiros de texto ou imagens.
- **Base de dados (SQLite e Room):** Para guardar listas de informação estruturada (como as tarefas da nossa aplicação).

1.1 Funcionamento da Biblioteca Room

O Room é a biblioteca recomendada pela Google para gerir a base de dados em aplicações Android. O seu funcionamento divide-se em três partes principais:

- **Entidade (@Entity):** Define a estrutura dos dados a guardar (por exemplo, os campos de uma tarefa).
- **DAO (@Dao):** Contém os métodos para realizar as operações CRUD (criar, ler, atualizar e apagar dados).
- **Base de Dados (@Database):** Gere a ligação e o acesso ao armazenamento no dispositivo.

1.2 Estrutura da Aplicação Desenvolvida

Na aplicação desenvolvida pelo nosso grupo, criámos um gestor de tarefas simples utilizando as três partes do Room:

- **TaskEntity (@Entity):** Representa a tabela da base de dados. Para cada tabela guarda um identificador (id), um título (title) e um valor booleano (isCompleted) para indicar se a tarefa está concluída ou não.

- **TaskDao (@Dao):** É a interface onde definimos as operações CRUD (Create, Read, Update, Delete) através de métodos Java associados a anotações do Room (@Insert, @Query, @Update, @Delete).
- **AppDatabase (@Database):** É a classe que define a base de dados Room ('task_database') e implementa o padrão Singleton, garantindo que a aplicação utiliza apenas uma única instância aberta da base de dados durante toda a sua execução.

1.3 Execução das Operações em Segundo Plano

No Android não é permitido fazer operações de base de dados (como ler ou gravar dados) no ecrã principal (UI Thread). Se fizermos isso, a aplicação pode travar ou dar erro como ANR (Application Not Responding).
Para resolver isto de forma simples em Java, utilizámos um ExecutorService:

- Quando o utilizador cria ou apaga uma tarefa, a gravação é feita numa tarefa em segundo plano (thread secundária).
- Assim que a gravação termina, usamos o método runOnUiThread() para atualizar a lista no ecrã.

2. Como Testar a Aplicação

Nesta secção descrevemos os passos necessários para compilar, executar e testar o funcionamento da aplicação e da persistência local de dados.

2.1 Objetivo

Verificar que as operações CRUD funcionam corretamente e que os dados adicionados permanecem guardados no dispositivo após fechar e voltar a abrir a aplicação sem ligação à Internet.

2.2 Requisitos

Para testar a aplicação é necessário ter o Android Studio instalado e utilizar um dispositivo com a versão Android 10 (API 29) ou superior. Todo o código-fonte da aplicação está disponível no repositório do GitHub.

2.3 Procedimento (Passo a Passo)

1. Clonar o repositório do GitHub e abrir o projeto no Android Studio.
2. Clicar em Run para instalar e abrir a aplicação no telemóvel ou emulador.
3. Inserir o nome de uma tarefa na caixa de texto e carregar em Adicionar para confirmar que aparece na lista.
4. Clicar na caixa de seleção (*CheckBox*) da tarefa para a marcar como concluída.
5. Desligar o telemóvel ou o emulador de qualquer conexão à internet, fechar completamente a aplicação e voltar a abri-la para verificar que a tarefa se mantém guardada.
6. Apagar a tarefa individualmente através do ícone do caixote do lixo.
7. Adicionar várias tarefas e carregar no botão “Apagar Tudo” para verificar que a lista fica vazia.

2.4 Resultados Esperados

- **Inserir tarefa com texto:** A tarefa é gravada na base de dados e surge imediatamente na lista.
- **Tentar inserir sem texto:** Surge o aviso "Por favor, escreve o nome da tarefa." e nada é gravado.
- **Marcar a caixa de seleção (*CheckBox*):** O estado de conclusão da tarefa é atualizado na base de dados.
- **Fechar e reabrir sem internet:** As tarefas inseridas mantêm-se guardadas e visíveis na lista.
- **Carregar em "Apagar Tudo":** Todas as tarefas são apagadas e a lista fica vazia.

2.5 Métricas

- **Persistência dos dados:** As tarefas mantêm-se guardadas e visíveis na aplicação após fechar e voltar a abrir a aplicação sem ligação à Internet.
- **Comportamento da aplicação:** A interface responde de imediato ao adicionar, alterar ou apagar tarefas, sem travar/congelar a aplicação.

3. Excertos em Kotlin

Como a nossa aplicação principal foi desenvolvida em Java, apresentamos de seguida as versões equivalentes da Entidade e do DAO escritas em Kotlin, que se encontram guardadas na pasta docs/excertos_kotlin/.

3.1 TaskEntity em Kotlin (Data Class)

Em Kotlin, utilizamos uma *data class*, o que torna o código muito mais curto do que em Java. Não é necessário escrever manualmente os construtores, os *getters* ou os *setters*, pois o Kotlin cria-os automaticamente.

```
@Entity(tableName = "tasks")
data class TaskEntity(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    @ColumnInfo(name = "title")
    val title: String,
    @ColumnInfo(name = "is_completed")
    var isCompleted: Boolean = false
)
```

3.2 TaskDao em Kotlin

A interface do DAO em Kotlin utiliza exatamente as mesmas anotações do Room (@Dao, @Insert, @Query, @Update, @Delete), mas aproveita a vantagem do Kotlin suportar nativamente valores nulos (como no método getTaskById, que devolve TaskEntity?).

```
@Dao
interface TaskDao {
    // CREATE: Insere uma nova tarefa e devolve o ID gerado
    @Insert
```

```
    fun insert(task: TaskEntity): Long
    // READ: Seleciona todas as tarefas ordenadas por ID decrescente
    @Query("SELECT * FROM tasks ORDER BY id DESC")
    fun getAllTasks(): List<TaskEntity>
    // READ: Seleciona uma tarefa pelo ID com segurança contra nulos
    @Query("SELECT * FROM tasks WHERE id = :taskId LIMIT 1")
    fun getTaskById(taskId: Int): TaskEntity?
    // UPDATE: Atualiza os dados de uma tarefa existente
    @Update
    fun update(task: TaskEntity)
    // DELETE: Elimina uma tarefa específica
    @Delete
    fun delete(task: TaskEntity)
    // DELETE: Elimina todas as tarefas da tabela
    @Query("DELETE FROM tasks")
    fun deleteAll()
}
```